

Desafio Técnico - Tech Artist

Árnilsen Arthur Castilho Lopes

Este documento resume as implementações que eu fiz no meu projeto como candidato para a vaga de Artista Técnico da Gazeus. Além dos efeitos, usei o projeto como ferramenta para demonstrar meu conhecimento e habilidade em diversas áreas e ferramentas diferentes dentro da Unity, desde desenvolvimento, criação, importação e otimização de assets, profiling, code analysis e até mesmo design. Na seção de **Destaque Gerais** eu dei foco a algumas habilidades / implementações que são válidas de serem destacar.

Importante; Todos os assets existentes no projeto foram desenvolvidos por mim, Árnilsen Arthur, com auxílio do figma para artes e para todo o resto usando JetBrains Rider e as ferramentas internas da unity de desenvolvimento. Apenas músicas, efeitos sonoros e a fonte utilizada (Comfy Feeling) não foram feitas por mim.

Resultado Final



<https://github.com/arnilsenarthur/desafio-match-3/tree/task/tech-art-challenge-arnilsen>

Adições de Gameplay

Além de uma reorganização completa do código, foram adicionados ao sistema de gameplay:

- Suporte a dificuldades
- Peças especiais (Coringa, Bomba e Skull/Crânio)
- Recorde
- Tempo limite de partida

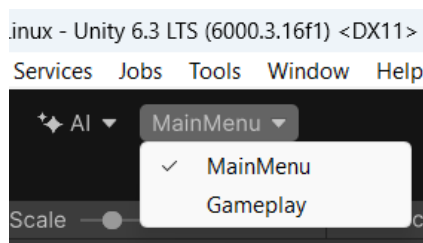
Adições Gerais

- Sistema de Idiomas
- Sistema de Configurações
- Sistema de Áudio (Música e Sons)
- Sistemas gerais de interface
- Sistemas de Pool para a performance
- Sistema de tutorial interativo

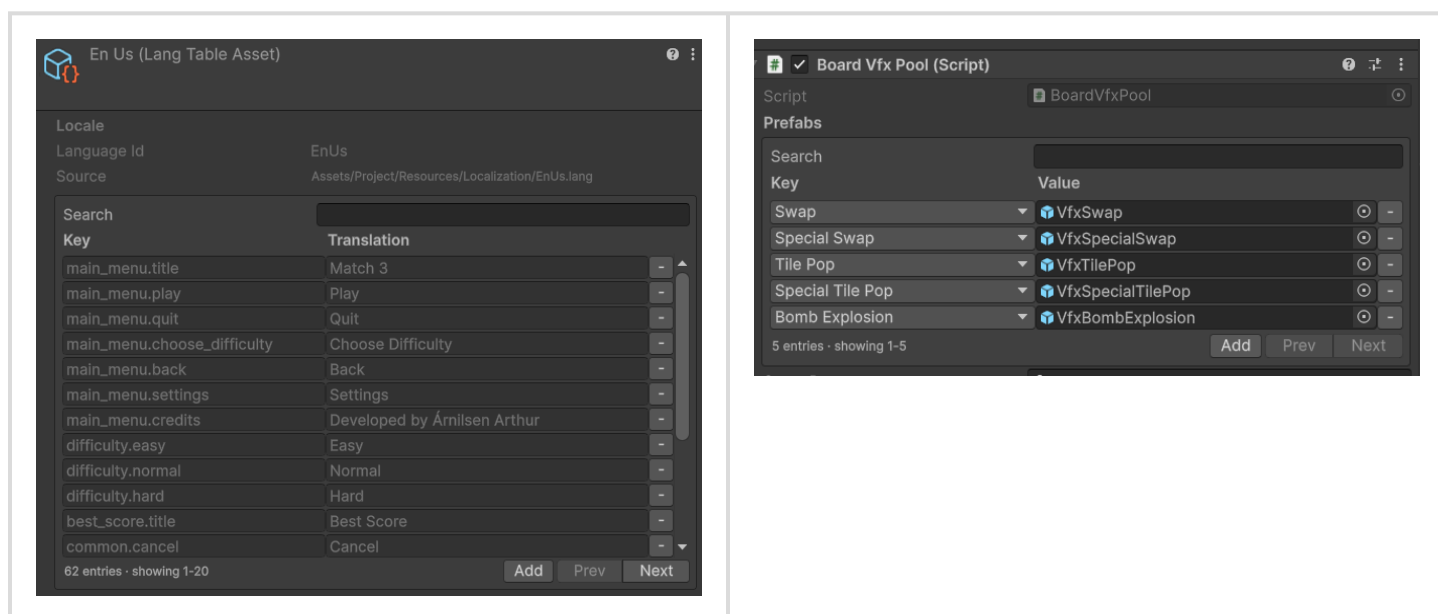
Destques Gerais

Ferramentas de Editor

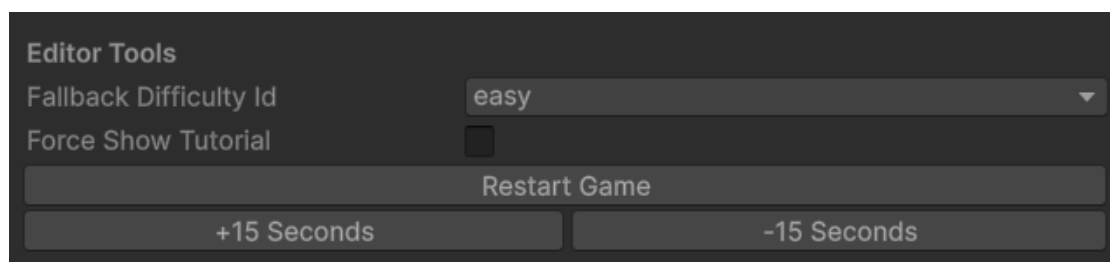
- Seletor de cena (para facilitar a navegação entre cenas durante o desenvolvimento)



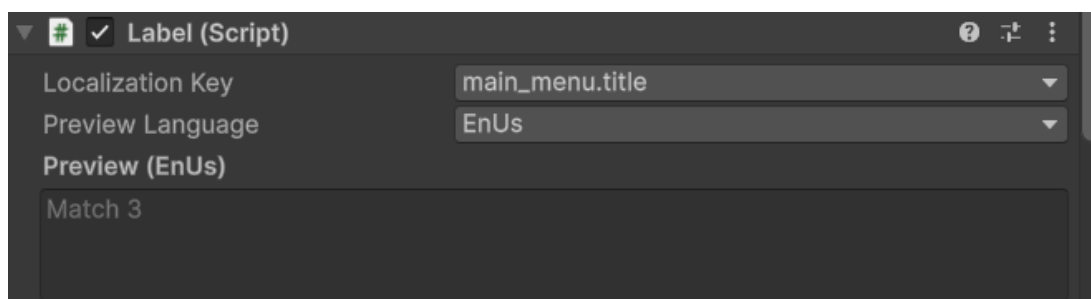
- PropertyDrawer customizado para o tipo SerializableDictionary, implementado como base para o sistema de idioma e registradores gerais, como os de prefabs de tiles e de prefabs de pools



- Controles rápidos de GameController para facilitar testes direto na cena, escolha de dificuldades, tutorial



- Preview de tradução para as Labels da interface



Importação de Áudios

Todos os áudios foram devidamente configurados, desativando o suporte a 3D tanto para os efeitos sonoros quanto pra música. Também, para os efeitos sonoros foi desligado o suporte a Estéreo, já que nenhum áudio utilizava. Foi configurada também a compressão das músicas para a redução do tamanho dos arquivos, tentando manter uma boa qualidade. Vale ressaltar que o sistema de áudio também dá suporte para que cada áudio tenha um volume base, para facilitar integrar vários audios no projeto

Audio Registry

Maps audio keys to clips. Use Desafio Match 3 > Audio > Rebuild Audio Registry to refresh from Project/Audio.

Search

Audio Key	Sound Entry									
ui.click	 AudioClick		Music	☐	Volume	1				
ui.popup	 AudioPopup		Music	☐	Volume	1				
ui.check	 AudioCheck		Music	☐	Volume	1				
ui.pause	 AudioPause		Music	☐	Volume	1				
gameplay.game_start	 AudioGameStart		Music	☐	Volume	1				
gameplay.tile_select	 AudioTileSelect		Music	☐	Volume	1				
gameplay.tile_slide	 AudioTileSlide		Music	☐	Volume	1				
gameplay.match	 AudioMatch		Music	☐	Volume	1				
gameplay.match_special	 AudioMatchSpecial		Music	☐	Volume	1				
gameplay.game_over	 AudioGameOver		Music	☐	Volume	1				
gameplay.high_score	 AudioHighScore		Music	☐	Volume	1				
ui.tutorial_page	 AudioTutorialPage		Music	☐	Volume	1				
music.main_menu	 MusicMainMenu		Music	☒	Volume	1				
music.gameplay	 MusicGameplay		Music	☒	Volume	1				
gameplay.explosion	 AudioExplosion		Music	☐	Volume	1				

15 entries · showing 1-15

Add

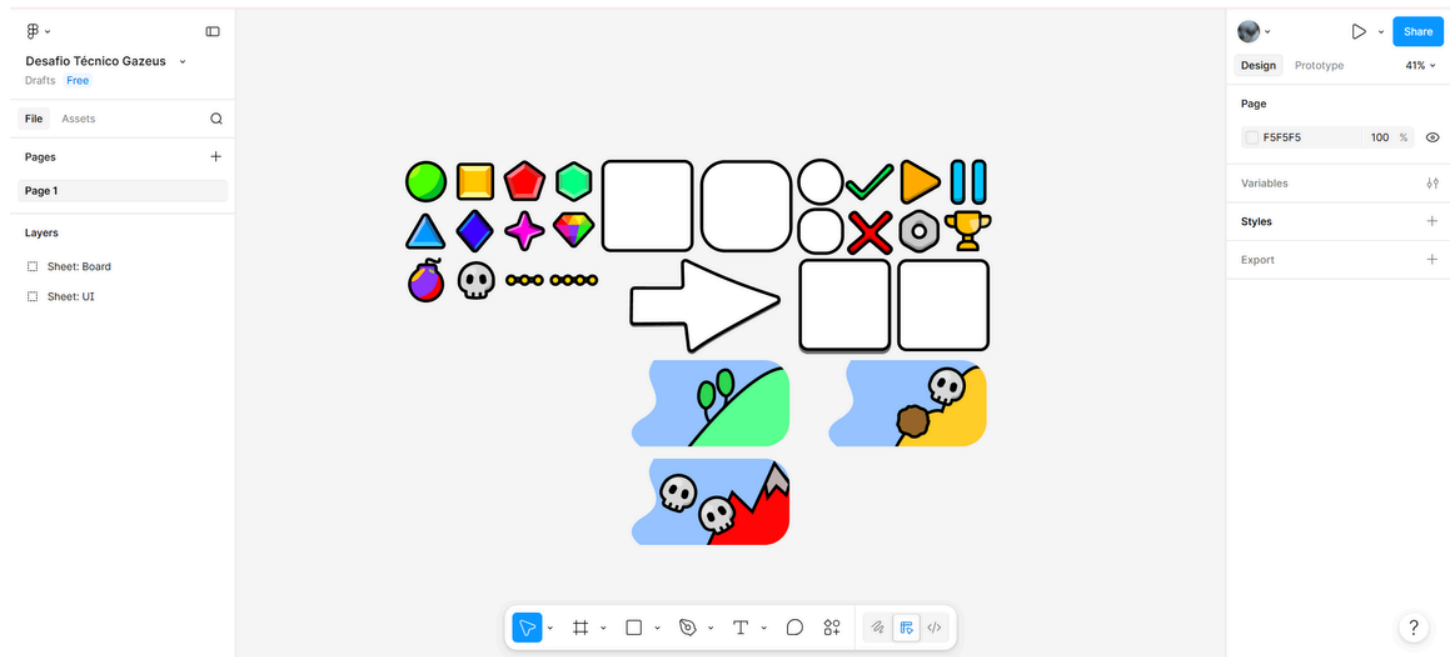
Prev

Next

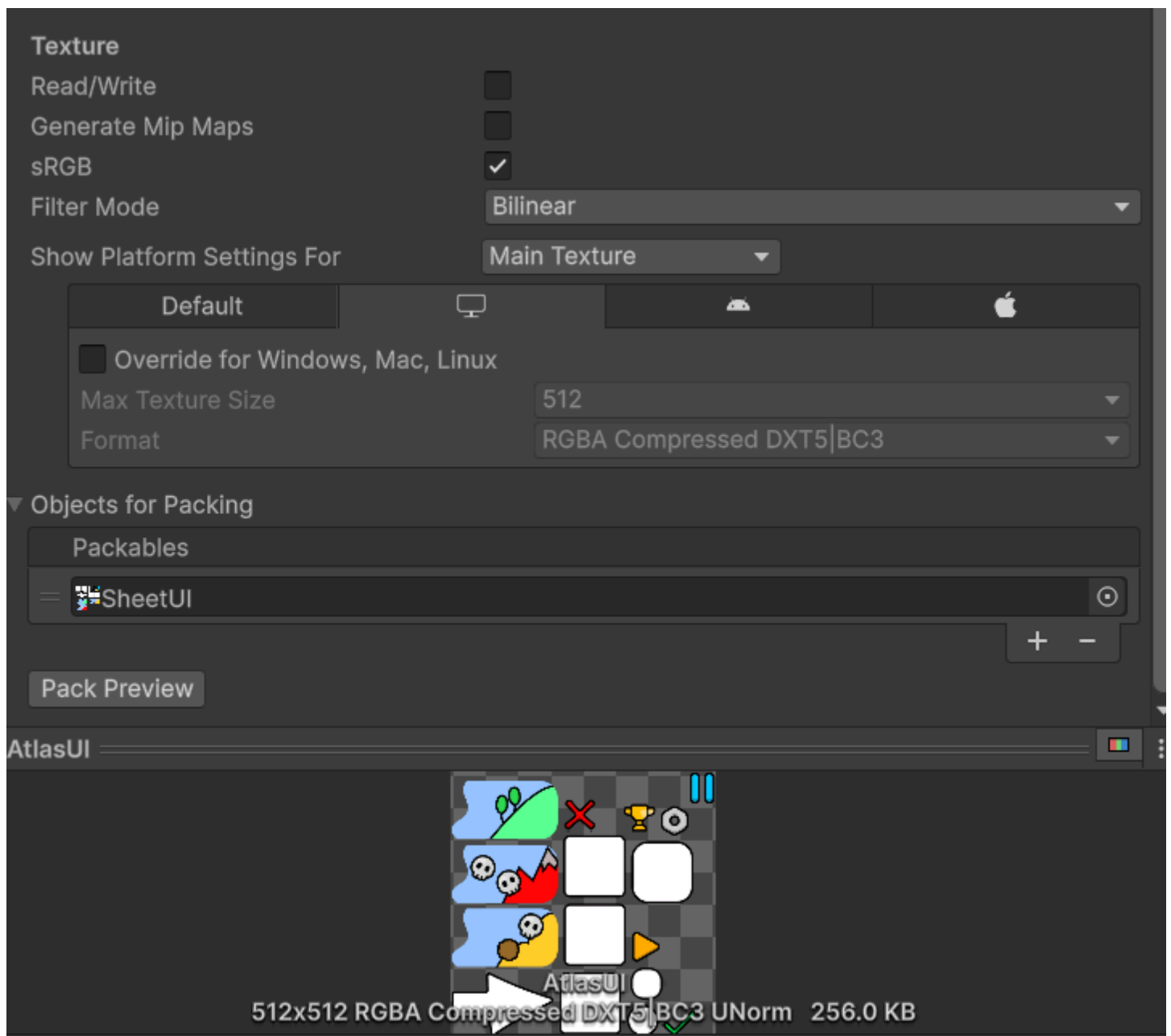
Importação de Sprites

Todos os sprites foram devidamente configurados devidamente, principalmente visando suporte a Android/iOS. Além disso foi utilizado o sistema de Sprite Atlas da unity para uma melhor exportação das texturas finais em build, visando melhor performance e melhor uso de memória. Também foi utilizado o pacote 2D Tooling da unity para verificar se os atlas teriam melhorias a serem feitas. No caso mantive os 3 atlas separados, pois não é uma boa prática agrupar os atlas de

Interface com Gameplay (já que os sprites de gameplay não são usados na cena de menu), assim como não seria boa prática agrupar os dos efeitos especiais.



Como fui eu quem criei os sprites já os criei pensando nas dimensões em que seriam usados, mas mesmo assim fiz questão de configurar os assets para a dimensão correta na unity, além de configurar o melhor formato de compressão para Android e iOS. Vale ressaltar que os sprites em si estão configurados sem compressão pois a compressão deve ser configurada apenas do atlas final (boa prática):

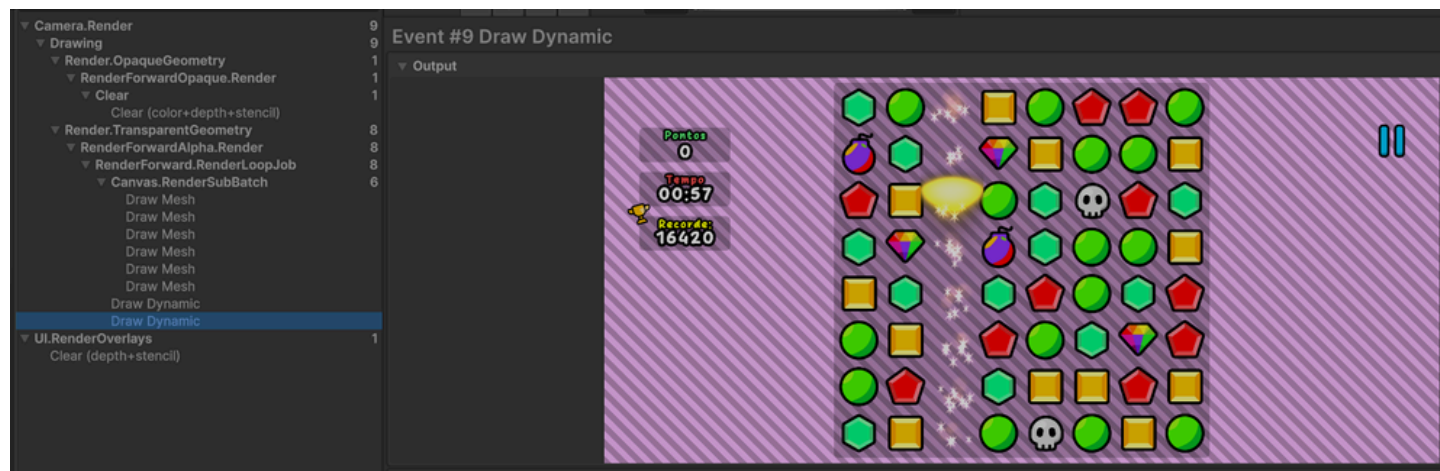


Sprite Atlas Analyzer												
Reports												
All Sprite Atlases												
	Name	Sprites	Total M...	Unused...	Usage	Total Ar...	Used A...	Width	Height	Pages	Packin...	Texture...
All Sprite Atlases	3											
Atlas Page Count > 1	0											
Source Textures with com...	0											
Texture Space Wastage > ...	0											
Sprite Count <= 1	0											
Textures contain different ...	0											
	> AtlasGai	12	256.0 KB	204.4 KB	20,2 %	524.288	105.739	512	512	1		DXT5
	> AtlasUI	16	256.0 KB	168.4 KB	34,2 %	524.288	179.492	512	512	1		DXT5
	> AtlasVfx	7	256.0 KB	196.1 KB	23,4 %	524.288	122.716	512	512	1		DXT5

Profiling de Renderização

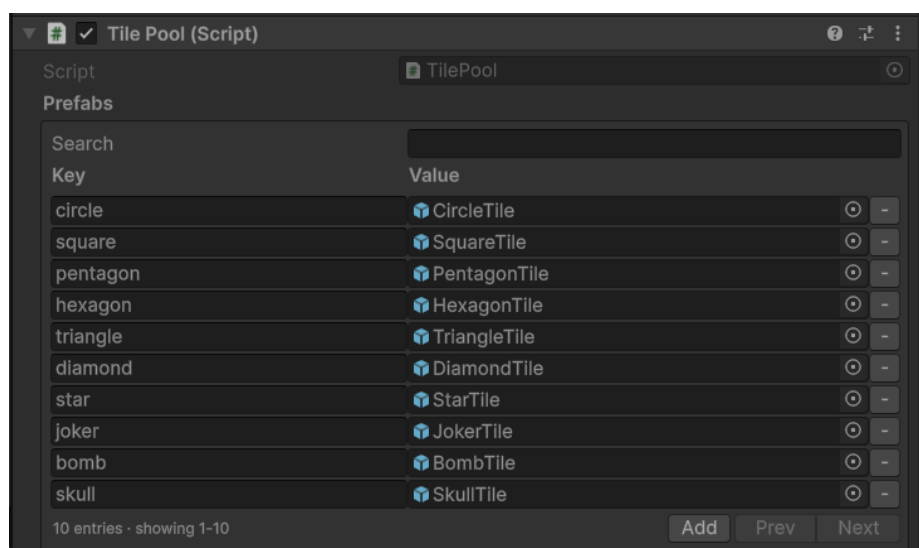
Pela forma com que os atlas dos sprites, os efeitos e as interfaces foram configuradas, o número de draw calls do projeto foi mantido bem reduzido. Todas as peças, todos os elementos de interface (exceto texto que tem sua própria Draw

Call) foram unificados cada grupo em um draw call apenas. A análise de overdraw até aconteceu mas é praticamente irrisória nesse caso. Os elementos de sobreposição na interface por causa do uso de transparência, que pelo tamanho do projeto não apresenta impacto significativo de performance.



Profiling de Performance / Código

O profiling não mostrou pontos críticos durante a gameplay, mas existiam dois pontos principais para se tomar cuidado na implementação: A instanciação de objetos (peças, efeitos e audio sources) e a lógica de matching / gameplay. Quanto a instanciação foi implementado um sistema completo de Pool com registros de prefabs.



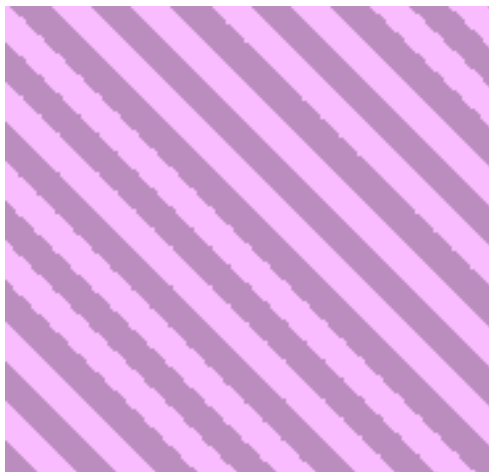
Quanto a lógica de matching das peças (e a gameplay de forma geral), melhorias foram sendo feitas durante o projeto para garantir uma boa qualidade e performance, enquanto os códigos e componentes se mantinha organizados e legíveis.

Tentei seguir o padrão de nomenclatura que já existia no repositório, apesar de divergir do que eu estive acostumado a usar nos últimos anos (pequenas

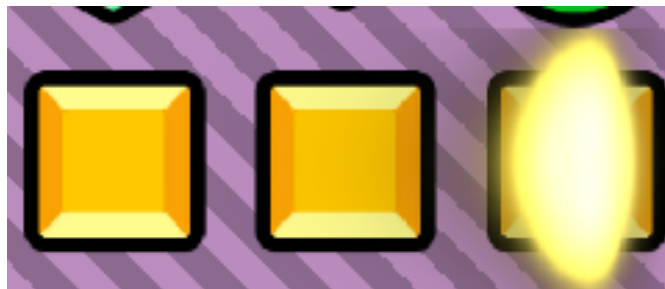
diferenças, como os campos serializáveis mesmo sendo privados sempre começarem sem o underline), e também tentei manter a mesma separação de componentes (View / Board).

Efeitos e Animações

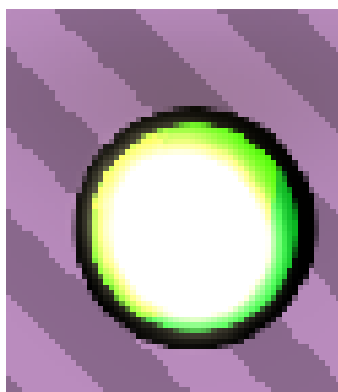
Como pedido alguns efeitos foram criados, misturando partículas, shaders e animações utilizando DoTween, sendo esses:



Shader de ScrollableBackground



Shader de Swipe Normal



Partícula de TilePop Normal



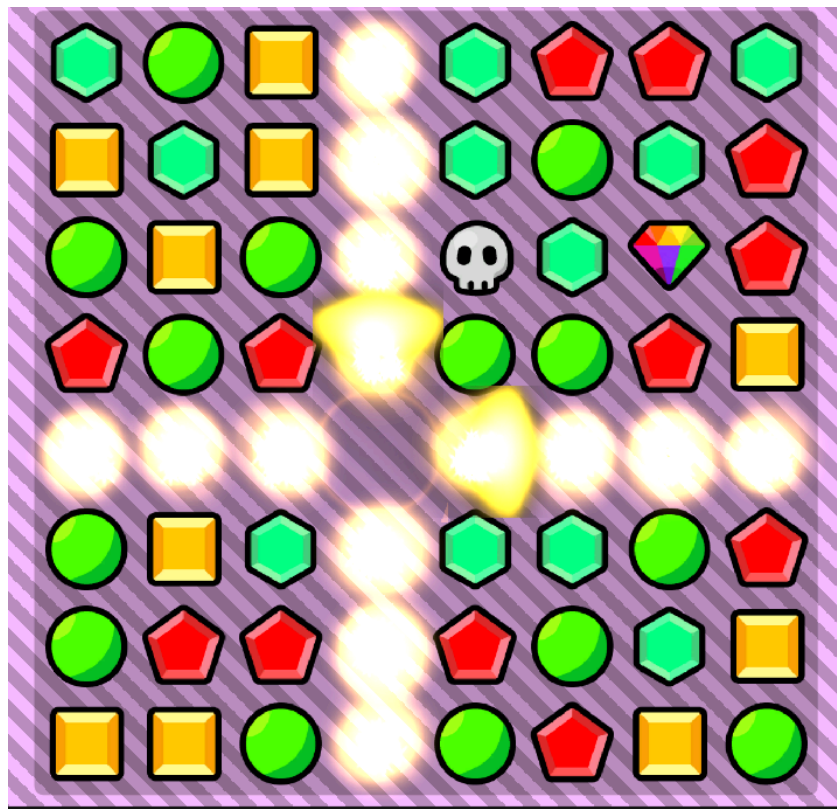
Shader de Swipe Especial



Particula de TilePop Special



Particula + Animação de Explosão



Arquitetura do Projeto

Usei dois princípios principais que valem serem ditos. O primeiro é um sistema de Serviços (Audio, Gameplay, Tradução, etc...) fazendo com que sistemas interajam entre si de forma mais tranquila, além de facilitar um pouco o desenvolvimento dos scripts. O segundo ponto foi basear todo o looping de gameplay em eventos, facilitando a implementação de efeitos sonoros, animação, da sequenciação das ações e também do tutorial, usando os callbacks para interceptar o que acontecia durante o jogo.

```
1 public static class GameService
2 {
3     private static GameState _state;
4     private static GameController _controller;
5
6     public static event Action<GameStartedEventArgs> GameStarted;
7     public static event Action<ScoreChangedEventArgs> ScoreChanged;
8     public static event Action<TimeChangedEventArgs> TimeChanged;
9     public static event Action<CascadeStepEventArgs> CascadeStep;
10    public static event Action<BoardRegeneratedEventArgs> BoardRegenerated;
11    public static event Action<GameEndedEventArgs> GameEnded;
12    public static event Action<CountdownChangedEventArgs> CountdownChanged;
13    public static event Action<TutorialGuideEventArgs> TutorialGuideChanged;
14
15    public static bool IsActive => _state != null;
16    public static bool IsGameOver => _state != null && _state.IsGameOver;
17    public static bool IsTutorialMode => _state != null && _state.IsTutorialMode;
18    public static bool IsPaused => _controller != null && _controller.IsPaused;
19    public static float TimeRemaining => _state?.TimeRemaining ?? 0f;
20    public static int Score => _state?.Score ?? 0;
21    public static BoardState Board => _state?.Board;
22 }
```

Também tomei cuidado com reload/recompilação para que o projeto não quebre / pare de funcionar quando você recompila, facilitando muito o desenvolvimento e os ciclos de teste.